Build a **production-ready full-stack trade journal web app** using **Next.js**, from initial project setup to final deployment readiness for **Vercel**. Use a **free PostgreSQL database service**, preferably **Neon**, with **Prisma ORM**. Implement the **full UI, backend, database, authentication, analytics, calendar, and seed data**. The app must be fully functional, modular, responsive, and cleanly structured for future scale.

The app is for **professional prop firm and live trading journaling**. It must feel like a **premium trading dashboard**, not a generic admin template.

CORE STACK

Use the following stack:

- Next.js latest with **App Router**
- TypeScript
- Tailwind CSS
- shadcn/ui
- Lucide React icons
- PostgreSQL with Prisma ORM
- Neon free Postgres
- Zod for validation
- JWT authentication
- HTTP-only cookies for auth token storage
- Vercel-ready deployment setup
- Seed script with realistic sample data

AUTHENTICATION REQUIREMENTS

Implement **JWT auth** with a clean, secure setup.

Requirements:

- Support:

    - Register
    - Login
    - Logout
    - Get current user session
- Use **JWT tokens stored in HTTP-only cookies**
- Do not store auth tokens in localStorage
- Use secure cookie settings appropriate for production
- Structure auth so it works well on Vercel
- Add route protection so journal pages require login
- Keep architecture ready for future enhancements like refresh tokens or password reset
- Hash passwords securely using bcrypt or bcryptjs
- Add a `User` model in Prisma
- Every trade and settings record must belong to a user
- Only show the logged-in user's trades, analytics, and phase data

PRODUCT GOAL

Build a **professional trade journal web app** that tracks trades by phase and automatically calculates all analytics from saved trades.

The app must support:

- trade journaling
- multi-phase tracking
- analytics
- calendar-based review
- structured multi-timeframe "reason for trade" builder

- dark/light theme
- future multi-user scalability

The UI must feel:

- premium
- serious
- clean
- trading-focused
- data-centric
- polished in both dark and light mode

IMPORTANT PRODUCT RULES

- All analytics must be derived automatically from stored trades
- No manual analytics entry
- Data must be separated by phase
- Historical phase data must remain stored in the database
- When switching between phases, the app must show only that phase's data
- New trades must automatically inherit the currently selected phase
- Calendar, summary cards, analytics, and master table must all reflect the selected phase
- Dark mode must remain highly readable everywhere
- Direction badges must remain visible in dark mode:
    - Long = green styling
    - Short = red styling

PHASE MODEL

Support these phases:

- Phase 1
- Phase 2
- Live

Behavior:

- Switching phase filters the whole app
- Trades remain stored historically
- New trades are created under the selected phase
- Each phase has its own analytics and totals
- Selected phase should persist across refresh, preferably per user

Default target logic:

- Phase 1 = 8% of starting balance
- Phase 2 = 5% of starting balance
- Live = 10% of starting balance

Make this configurable in code and stored in phase settings so it can be changed later.

APP STRUCTURE

Create the following main structure:

Public/Auth Pages

- Login page
- Register page

Protected App Area

Main authenticated dashboard with:

- Header
- theme toggle
- phase selector
- top summary cards
- tabbed layout or segmented layout with:
    - Master Table
    - Analytics & Calendar

Use a polished app shell with excellent spacing, rounded cards, soft shadows, modern typography, and strong dashboard hierarchy.

DASHBOARD HEADER

Include:

- App title: **Trade Journal**
- User menu or profile area
- Logout action
- Theme toggle
- Clean responsive layout

TOP SUMMARY CARDS

These must always reflect the currently selected phase for the logged-in user.

Show:

- Starting Balance
- Current Balance
- Phase Target
- Total PnL

- Win Rate
- Profit Factor
- Returns
- Average R:R
- Remaining to Goal
- Remaining to Phase Target

These must update automatically when trades are added, edited, deleted, or when phase changes.

MAIN APP TABS

Create two main sections:

1. Master Table
2. Analytics & Calendar

MASTER TABLE SECTION

This section contains:

A. Trade Entry Form

Fields:

- Trade No. auto-generated
- Date, default to current date
- Direction: Long / Short
- PnL
- Setup Category: Cont / Reversal

- Bias: Bullish / Bearish / Range
- R:R Ratio
- Additional Notes
- Phase should be automatically derived from selected phase and not manually chosen in the form

Validation requirements:

- Require at least:
    - PnL
    - one valid reason selection
- Validate with Zod
- Show clean inline error states
- Reset form properly after successful save

B. Trade Reason Builder

This is a critical feature and must be implemented carefully.

Build a **multi-timeframe reason builder** with tabs:

- 4H
- 1H
- 15M
- 5M

Under each timeframe tab include:

1. Bias selector

Options:

- Bullish

- Bearish
- Range

2. Confluence checklist options

Default options:

- Liquidity Sweep
- FVG
- OB
- BOS
- CHOCH
- Breaker
- Mitigation Block
- Premium/Discount
- Discount
- Premium
- Session Setup
- Trend Continuation

3. Custom confluence input

- Small text input
    - button
- When user types a custom confluence and clicks +, add it to selectable list
- Custom confluences must be removable with a delete button
- If a custom confluence is deleted, remove it from current form selections too

4. Live reason summary panel

Show a generated “Reason for Trade” summary beside or below the builder.

Example output:

- 4H: Bullish, FVG, Discount
- 1H: Bullish, FVG, BOS
- 15M: Bullish, BOS, Mitigation Block, OB
- 5M: BOS, FVG

Reason Data Storage Requirements

Store both:

1. **Normalized structured data** in the database
2. **Readable summarized reason strings** for quick rendering

Recommended structure for normalized storage:

- `reasonStructure` as JSON object with keys:
    - 4H
    - 1H
    - 15M
    - 5M
- Each timeframe should include:
    - bias
    - selected confluences

Example concept:

- 4H → bias: Bullish, confluences: [FVG, Discount]
- 1H → bias: Bullish, confluences: [FVG, BOS]

Also persist summarized readable reasons for easy table display.

C. Add Trade Action

- Save trade to DB
- Validate form using Zod
- Save under current phase automatically
- Reset form cleanly after submit
- Update master table, analytics, and calendar immediately after save

D. Master Trades Table

Columns:

- Trade No
- Date
- Direction
- PnL
- Setup Category
- Bias
- R:R
- Reasons
- Notes
- Phase
- Action

Actions:

- Delete trade

- Edit trade
- Prefer edit via modal or side panel
- Deleting or editing must immediately update analytics and calendar

Direction badge styling:

- Long badge = green
- Short badge = red
- Must remain readable in both light and dark themes

Add clean table states for:

- empty state
- loading state
- error state

Optional filters if time allows:

- date range
- direction
- setup category
- phase-aware search

ANALYTICS & CALENDAR SECTION

This section must reflect only the selected phase for the logged-in user.

A. Analytics Summary Cards

Compute automatically from saved trades only:

- Win Rate
- Total PnL
- Returns
- Profit Factor
- Average R:R

B. Direction-Based Analytics Tables

Two side-by-side tables:

- Long analytics
- Short analytics

Each table must show:

- Total Trades
- Wins
- Losses
- Win %
- Average Win
- Average Loss
- Average R:R

All values derived from the stored trades.

C. Goal Tracking Section

Show:

- Set Goal
- Current PnL

- Required to Reach Goal
- Phase Target Left
- Progress bar

Requirements:

- Goal should be editable
- Prefer storing goal per phase per user
- If needed, create a `PhaseSettings` or `JournalSettings` table tied to user + phase
- Make future expansion easy

D. Real Monthly Calendar UI

Build a real calendar component, not a simple list.

Requirements:

- Sunday to Saturday columns
- Additional **Summary** column at the end of each week row
- Month-by-month navigation
- Previous / next month buttons
- Optional month dropdown jump
- Show one month at a time
- Do not make months appear in one endless stacked view

Inside each day cell show:

- number of trades that day
- daily PnL

Layout requirements:

- Center trade count and PnL neatly in the cell
- Positive day styling = green
- Negative day styling = red
- Neutral day styling = muted
- Non-month dates should appear visually muted

Weekly summary column must show:

- total number of trades that week
- total weekly PnL

Month header must show:

- Month and year
- Month P/L
- Month trade count
- Win Rate
- Profit Factor

Calendar data must reflect:

- selected phase
- selected month
- current logged-in user only

DARK MODE / LIGHT MODE

Implement a strong theme system.

Requirements:

- Fully readable dark mode everywhere
- Proper contrast in:
  - cards
  - tables
  - tabs
  - calendar
  - badges
  - forms
  - dropdowns
  - modals
- Avoid weak gray-on-gray styling
- Use premium dark surfaces
- Long/Short badges must remain visible in dark mode
- Persist theme choice across refresh

DATABASE DESIGN

Create a full Prisma schema with migrations.

Use a structure similar to this, but improve if needed while preserving all requirements.

User

Fields:

- id
- email

- passwordHash
- name optional
- createdAt
- updatedAt

Trade

Fields:

- id
- tradeNumber
- date
- direction
- pnl
- setupCategory
- bias
- rr
- notes
- phase
- summarizedReasons
- reasonStructure
- createdAt
- updatedAt
- userId

PhaseSettings or JournalSettings

Fields:

- id
- userId
- phase
- startingBalance
- goal
- phaseTargetPercent
- createdAt
- updatedAt

Make sure:

- every trade belongs to a user
- tradeNumber generation is deterministic and safe
- if trade numbers are per user and phase, implement that clearly
- schema is future-proof for auth and scaling

Use Prisma enums where appropriate for:

- direction
- setupCategory
- bias
- phase

Use JSON type where appropriate for structured reasons.

BACKEND REQUIREMENTS

Implement full backend functionality.

Required capabilities:

- register user
- login user
- logout user
- fetch current user
- create trade
- edit trade
- delete trade
- fetch trades by selected phase
- fetch trades by phase and month
- fetch analytics by phase
- fetch calendar aggregation by phase and month
- fetch and update phase settings

Implementation guidance:

- Use route handlers and/or server actions appropriately
- Keep server/client boundaries clean
- Use Zod validation
- Use Prisma transactions where appropriate
- Protect authenticated routes
- Keep backend modular and typed

PERFORMANCE REQUIREMENTS

Do not calculate everything repeatedly on the client.

Requirements:

- analytics calculations should be implemented in a shared server-side utility layer or backend service layer
- calendar aggregation should be done server-side and returned in a shape optimized for rendering
- avoid inefficient repeated client-side recalculation
- use React Server Components where appropriate
- minimize unnecessary `use client`
- keep client components focused on interaction-heavy UI only

---

UI / UX REQUIREMENTS

The app must have a premium trading-dashboard aesthetic.

Requirements:

- polished dashboard feel
- strong spacing and hierarchy
- rounded cards
- soft shadows
- modern typography
- responsive design
- desktop-first but mobile-friendly
- clean states for loading, empty, and error UI
- avoid clutter
- reason builder must feel intuitive and not cramped

- calendar must feel polished and serious

Do not generate a toy mockup. Build the actual working app.

COMPONENT ARCHITECTURE REQUIREMENTS

Use clean reusable components.

Separate components for:

- app shell
- auth forms
- summary cards
- phase selector
- analytics cards
- analytics tables
- goal tracker
- calendar
- trade form
- reason builder
- trade table
- edit trade modal or drawer
- theme toggle
- user menu

Avoid monolithic code.

Use clear naming and clean folder structure.

Use comments only where genuinely helpful.

SUGGESTED FOLDER ORGANIZATION

Use a maintainable structure similar to this:

- `app/`
  - auth routes
  - dashboard routes
  - API route handlers if needed
- `components/`
  - ui
  - auth
  - dashboard
  - journal
  - analytics
  - calendar
- `lib/`
  - auth
  - prisma
  - validations
  - analytics
  - calendar
  - utils

- `prisma/`
    - schema
    - seed
- `types/`

You may refine the exact structure, but keep it organized and scalable.

SEED DATA REQUIREMENTS

Include seed script with realistic sample data:

- demo user
- phase settings for Phase 1, Phase 2, and Live
- realistic trades across multiple dates and multiple phases
- examples of long and short trades
- examples with reason structure populated
- enough data to make calendar and analytics sections meaningful

---

VERCEL / DEPLOYMENT REQUIREMENTS

Make the app ready for deployment on Vercel.

Requirements:

- use environment variables correctly
- include `.env.example`

- configure Prisma for Neon correctly
- ensure route handlers/server actions work in Vercel environment
- include production-safe cookie/auth handling
- include instructions for:
    - local development
    - database setup
    - Prisma migrate
    - Prisma generate
    - Prisma seed
    - Vercel deployment

If connection pooling is needed for Neon, configure it correctly.

README REQUIREMENTS

Create a complete README including:

- project overview
- stack used
- local setup
- environment variables
- Neon database setup
- Prisma commands
- seed instructions
- how auth works
- how to run locally

- how to deploy to Vercel
- known extension points for future improvements

OPTIONAL NICE-TO-HAVES IF TIME ALLOWS

Implement these if possible after core requirements are complete:

- edit trade modal
- date range filtering
- direction filtering
- setup category filtering
- export to CSV
- selected phase persistence
- theme persistence
- month dropdown jump navigation
- separate editable goal per phase
- small dashboard animations only if subtle and professional

IMPORTANT IMPLEMENTATION RULES

- Do not build a fake UI-only prototype
- Build the real app with working backend and database integration
- All analytics must derive from stored trades

- All pages and data must be user-scoped
- Keep code production-oriented and maintainable
- Prefer correctness, architecture, and polish over unnecessary complexity
- Make the generated UI closely reflect these product rules and interactions

---

DELIVERY EXPECTATION

Generate the complete project with:

1. Next.js app
2. Prisma schema and migrations
3. Neon integration
4. JWT auth with HTTP-only cookies
5. Protected app routes
6. Seed script
7. Working dashboard UI
8. Trade CRUD
9. Analytics and calendar aggregation
10. Vercel-ready configuration
11. Complete README